

Problem

The 3N+1 sequence is generated using the following rules:
If n is even, the next number is $n / 2$. If n is odd, the next number is $3 \times n + 1$. The sequence continues until it reaches 1. The cycle length of a number is the total number of terms in the sequence, including the starting number and 1.

Given two integers i and j, determine the maximum cycle length among all integers between i and j (inclusive).

If $i > j$, swap them before processing.

Input Format

A single line containing two space-separated integers:

i j

Constraints

$1 \leq i, j \leq 1,000,000$

Output Format

Print three integers:

i j maximum_cycle_length

using the original values of i and j.

Sample Input 0

1 10

[Change Theme](#)

Language

Java 15



```

1  import java.io.*;
2  import java.util.Scanner;
3  public class Solution {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          if (!sc.hasNextLong()) return;
7          long originalI = sc.nextLong();
8          long originalJ = sc.nextLong();
9          long start = originalI;
10         long end = originalJ;
11         if(start>end) {
12             long temp = start;
13             start=end;
14             end=temp;
15         }
16         long maxCycle=0;
17         for(long num=start;num<=end;num++) {
18             long current=num;
19             long length=1;
20             while(current!=1) {
21                 if(current % 2==0) {
22                     current=current/2;
23                 }else {
24                     current=3*current+1;
25                 }
26                 length++;
27             }
28             if(length>maxCycle) {

```

Line: 31 Col: 10